Big Data — 4-Mark Answer Set (12 Questions)

Each answer is sized for 4 marks: a clear opening definition + key points, no over-explanation. Diagram notes are flagged where they'd earn extra marks.

Q1 — What is Big Data? Explain its characteristics.

Big data refers to datasets so large, fast-moving, and varied that traditional database tools can't efficiently capture, store, or process them. It is defined by the **4 V's**:

- **Volume** — massive scale of data generated (e.g., by social media, sensors)
- **Velocity** — speed at which data is generated and must be processed
- **Variety** — structured, semi-structured, and unstructured data mixed together
- **Veracity** — trustworthiness/quality of the data

Example: Platforms like Twitter and Facebook generate terabytes of varied, fast-arriving data daily.

> 📊 Diagram: Not needed.

Q2 — What is Hadoop? Explain its core components and Ecosystem.

Hadoop is an open-source Apache framework for distributed storage and processing of large datasets across clusters of commodity hardware. It works on the principle of moving computation to the data rather than the reverse.

Core components:

- **HDFS** — distributed storage layer
- **MapReduce** — distributed processing layer
- **Hadoop Common** — shared utilities
- **YARN** — resource management/job scheduling

Ecosystem: Tools built around this core include HBase, Sqoop, Flume, Spark, Pig, Hive, Impala, Oozie, and Hue — each handling ingestion, querying, or scheduling tasks.

> 📊 Diagram: Recommended — simple layered stack (HDFS+YARN → MapReduce/Spark → Hive/Pig/HBase etc.).

Q3 — Explain HDFS architecture with advantages and limitations of Hadoop.

HDFS follows a **master-slave architecture**: a single **NameNode** manages metadata (filenames, block locations), while multiple **DataNodes** store actual data, split into 128MB blocks and replicated (typically 3x) for fault tolerance.

Advantages: fault-tolerant, high-throughput for large datasets, works on cheap commodity hardware, scales horizontally.

Limitations: inefficient for many small files, NameNode was a single point of failure (in older versions), not suited for low-latency real-time access.

> 📊 Diagram: Recommended — NameNode/DataNode layout with block replication.

Q4 — Explain how MapReduce works, including the role of a Combiner.

MapReduce processes large datasets in parallel across a cluster in three phases:

1. **Map** — reads input, emits intermediate key-value pairs
2. **Shuffle & Sort** — groups values by key, sorts them
3. **Reduce** — aggregates values per key into final output

Combiner: an optional local "mini-reducer" that runs on the mapper's output before shuffling, reducing the data sent over the network and speeding up the job.

> 📊 Diagram: Recommended — Map → Shuffle/Sort → Reduce flow.

Q5 — How does MapReduce/Hadoop handle node failures?

Hadoop handles failure in software rather than relying on fault-tolerant hardware, using four mechanisms:

- **Block replication** — each block copied to multiple DataNodes
- **Heartbeat monitoring** — missed heartbeats mark a node dead, triggering re-replication
- **Task re-execution** — failed Map/Reduce tasks are rescheduled on another node
- **Dynamic cluster membership** — nodes can join/leave without stopping the cluster

> 📊 Diagram: Not needed.

Q6 — Explain Matrix-Vector Multiplication using MapReduce.

To compute $x = Mv$ where $x_i = \Sigma_j m_{ij} \cdot v_j$:

- **Map:** for each matrix element m_{ij}, emit $(i, m_{ij} \times v_j)$
- **Shuffle:** group all values by key i
- **Reduce:** sum all values for each key i to produce (i, x_i)

If v is too large for memory, it is split into stripes matching column-stripes of M, so each mapper handles only its needed portion.

> 📊 Diagram: Recommended — small worked matrix-vector example.

Q7 — Explain Selection, Projection, Union, Intersection, and Difference using MapReduce.

Operation	Map	Reduce
Selection (σ)	Emit tuple if it passes the condition	None (identity)
Projection (π)	Emit required columns as key & value	Remove duplicate tuples
Union	Emit tuples from both relations	Output each distinct tuple once
Intersection	Tag tuple with its source relation	Output tuple only if from both relations
Difference (R−S)	Tag tuple with its source relation	Output tuple only if in R and not S

> 📊 Diagram: Optional.

Q8 — What is NoSQL? Explain its business drivers.

NoSQL is a non-relational database model built for horizontal scalability, using a dynamic (schema-less) structure and avoiding costly joins — unlike SQL's table-based, vertically-scaled, strict-ACID design.

Business drivers:

- **Volume** — data too large for a single machine
- **Velocity** — data arriving faster than one processor can handle
- **Variability** — data too inconsistent for a rigid schema
- **Agility** — need to change structures quickly without costly migrations

> 📊 Diagram: Not needed.

Q9 — Explain the NoSQL data architecture patterns.

Pattern	Description	Example
Key-Value store	Unique key mapped to a value; fast lookups	Redis, DynamoDB
Document store	Semi-structured JSON/BSON documents, dynamic schema	MongoDB
Column-family store	Data grouped by column families, write-heavy/sparse	Cassandra, HBase
Graph store	Nodes and edges, optimized for relationships	Neo4j

> 📊 Diagram: Recommended — one small sketch per pattern or a 4-way comparison.

Q10 — What is shared-nothing architecture? How does it help analyze Big Data?

Shared-nothing architecture is a distributed design where every node has its own independent CPU, memory, and disk — nothing is shared, and each node handles its own shard of data.

Helps Big Data by: enabling near-linear horizontal scalability, removing resource-contention bottlenecks, and providing natural fault isolation. Used in Cassandra, HBase, and Bigtable.

> 📊 Diagram: Recommended — independent nodes with no shared disk/memory.

Q11 — Differentiate between Master-Slave and Peer-to-Peer distribution models.

Aspect	Master-Slave	Peer-to-Peer
Writes	Only master accepts writes	Any node accepts writes
Coordination	Centralized	Decentralized
Single point of failure	Yes (master)	No
Example	HDFS, MongoDB (primary-replica)	Cassandra (ring topology)

> 📊 Diagram: Recommended — tree topology vs. ring topology.

Q12 — Explain the four ways NoSQL systems handle Big Data problems.

- **Move the query to the data** — query sent to nodes holding relevant data, instead of pulling data centrally
- **Hash rings** — consistent hashing distributes data evenly across nodes
- **Replication** — live backup copies allow read traffic to be spread across nodes
- **Distributed query aggregation** — the database handles running and combining query results across nodes

> 📊 Diagram: Not needed.

Diagram Summary

Q	Diagram Needed?
Q1	❌
Q2	✅ Ecosystem stack
Q3	✅ NameNode/DataNode
Q4	✅ Map-Shuffle-Reduce flow
Q5	❌
Q6	✅ Worked example
Q7	⚠️ Optional
Q8	❌
Q9	✅ Pattern comparison
Q10	✅ Independent nodes
Q11	✅ Tree vs. ring
Q12	❌